



UIT-HCM



Chương 7: Đa vi xử lý (Multiprocessors)

Computer Engineering Faculty

Embedded Systems

Nội Dung



1

Tổng quan về Multiprocessor

2

Kiến trúc phần cứng

3

Các vấn đề liên quan

4

Ví dụ



1. TỔNG QUAN VỀ MULTI-PROCESSOR

Tại sao là đa vi xử lý?



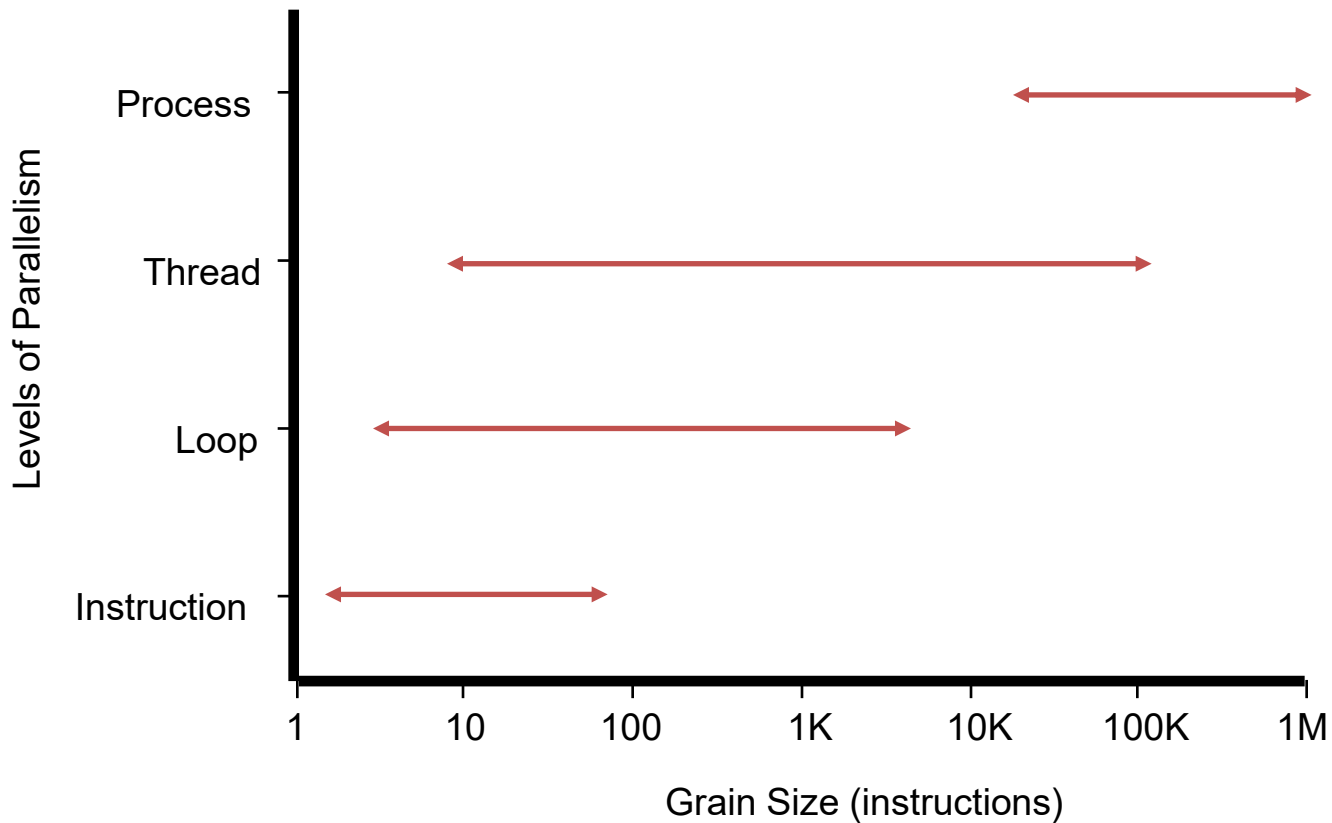
❑ Nhu cầu thực tế:

- Nhu cầu không ngừng của các hệ thống hiệu suất cao và cao hơn như Server, Network, Mobile app...
- Ham muốn sản phẩm thương mại khác biệt: nhanh hơn, mạnh hơn
- Gia tăng mức độ phức tạp và đa dạng của ứng dụng

❑ Cơ hội để phát triển đa vi xử lý:

- Sự phát triển của công nghệ bán dẫn: 130nm, 90nm, 45nm, 15nm...
- Sự phổ biến của máy tính

Mức độ xử lý song song



Đa vi xử lý cho bài toán xử lý song song

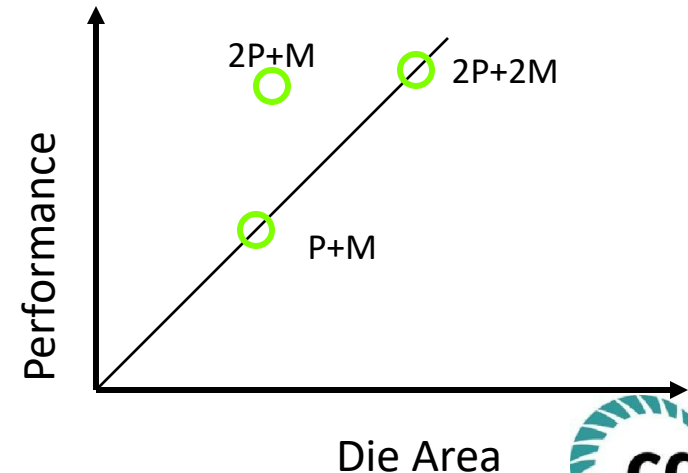
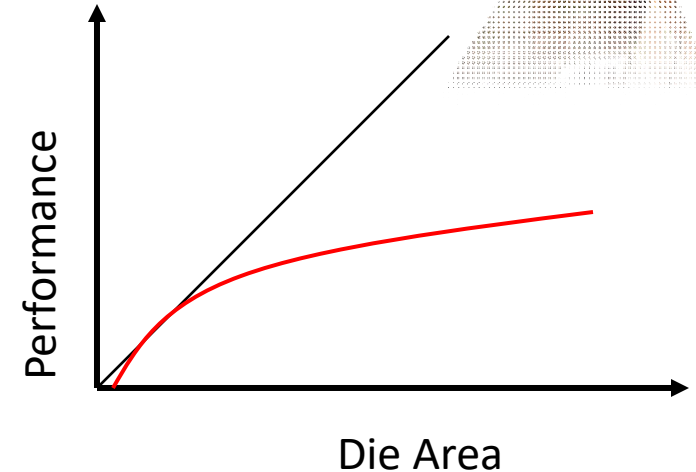


❑ ILP và LLP

- ILP và LLP còn hạn chế trong chương trình
- Tăng mức độ ILP thì “đắt đỏ”

❑ Hiệu suất tăng tuyến tính với xử lý song song?

- ## ❑ Thêm processor không đắt đỏ
- Nhưng hầu hết kèm thêm tăng bộ nhớ



Mục đích của đa vi xử lý



❑ Cải thiện năng lực tính toán chung:

Amdahl's law

$$s = \frac{1}{(1 - p) + \frac{p}{n}}$$

Trong đó:

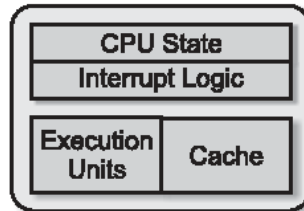
- s : Hệ số tăng tốc
- p : Tỷ lệ xử lý song song
- n : Số lượng đơn vị xử lý song song (số lượng processor)

❑ Nâng cao hiệu quả xử lý ứng dụng đa dạng và chuyên biệt:

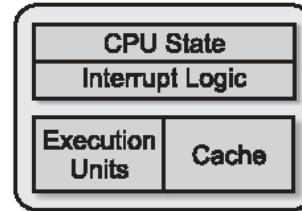
- Sự đa dạng của ứng dụng
- Sự đa dạng và linh hoạt trong kiến trúc đa vi xử lý

❑ Nâng cao độ tin cậy/độ sẵn sàng của hệ thống

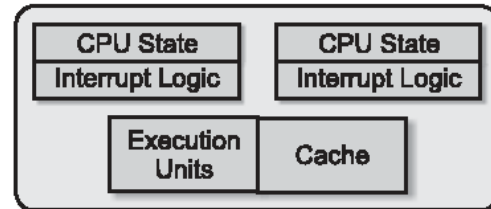
Multiprocessor và Multicore



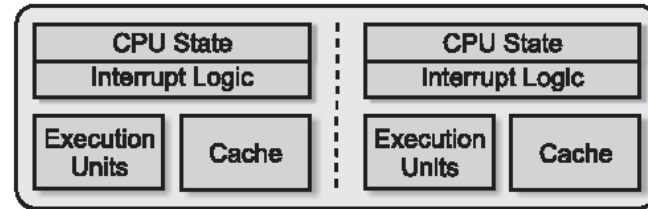
A) Single Core



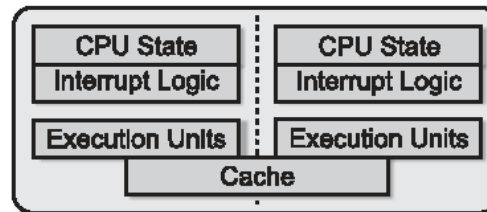
B) Multiprocessor



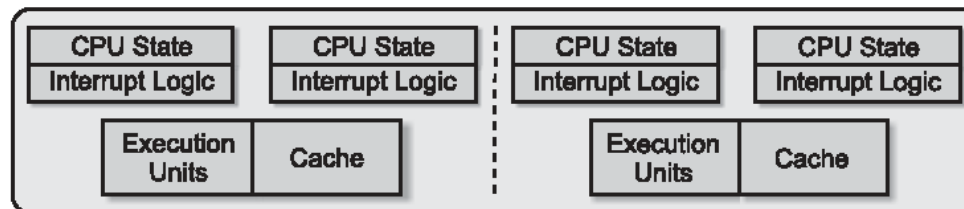
C) Hyper-Threading Technology



D) Multi-core



E) Multi-core with Shared Cache



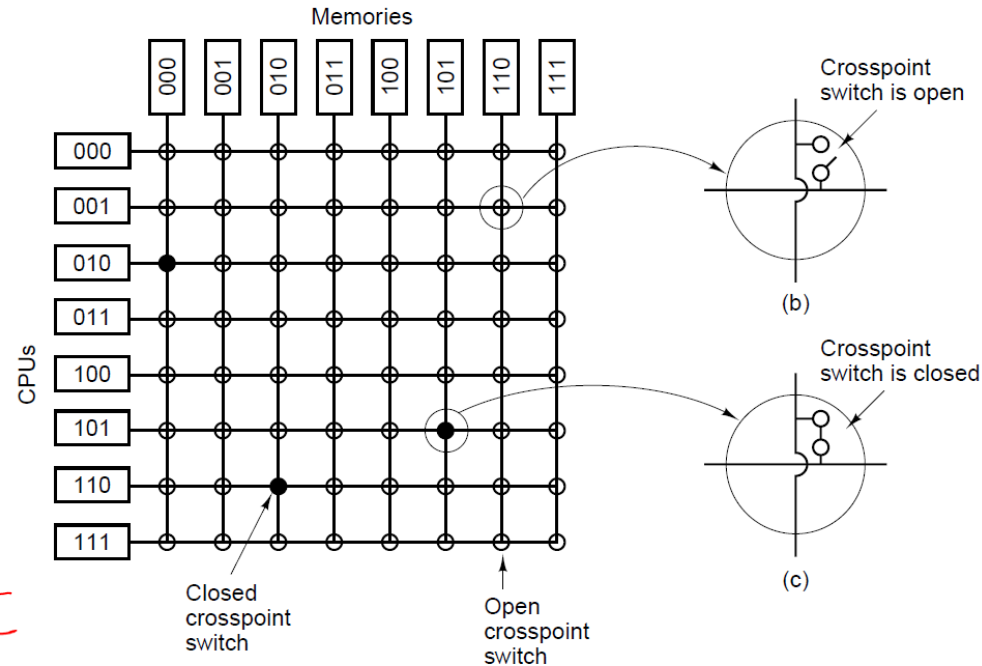
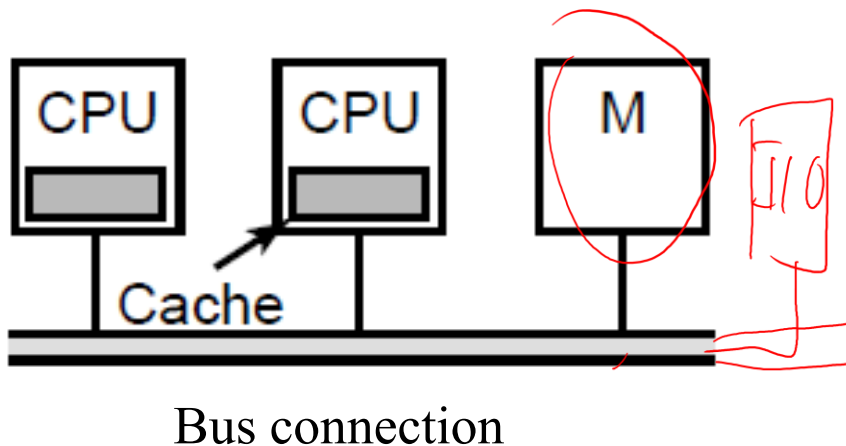
F) Multi-core with Hyper-Threading Technology

Các vấn đề với đa vi xử lý



❑ Mô hình kết nối giữa các processor (Topology):

- Bus
- Full-crossbar switch
- Mesh
- Torus
- ...



Các vấn đề với đa vi xử lý (tt)



- ☐ Giao tiếp giữa các processor (communication)
- ☐ Vấn đề điều khiển và dữ liệu:
 - Dòng lệnh được thực thi như thế nào
 - Dữ liệu được truy xuất ra sao
 - Đồng bộ hóa dữ liệu
- ☐ Phân chia tác vụ và di chuyển tác vụ giữa các processor
- ☐ Hệ điều hành trong hệ thống đa vi xử lý
- ☐ Khả năng tái cấu trúc hệ thống?



2. KIẾN TRÚC PHẦN CỨNG

- ❑ **UMA (Uniform Memory Access)** multiprocessors
- ❑ **NUMA (Nonuniform Memory Access)** multiprocessors

UMA Multiprocessor



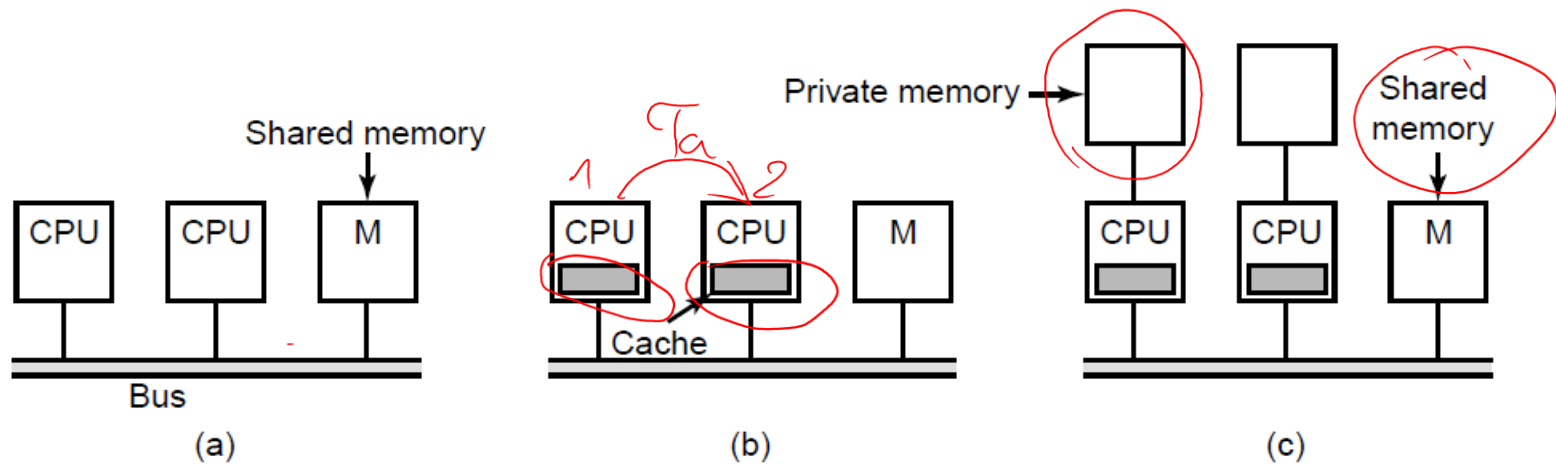
Bus-based multiprocessors

❑ Chia sẻ bus → tranh chấp bus

→ Băng thông bị hạn chế khi số lượng processor tăng lên

→ Cache → Đồng nhất cache (Cache coherence)

❑ Bao gồm bộ nhớ chia sẻ và cả bộ nhớ riêng trên processor



Three bus-based multiprocessors.

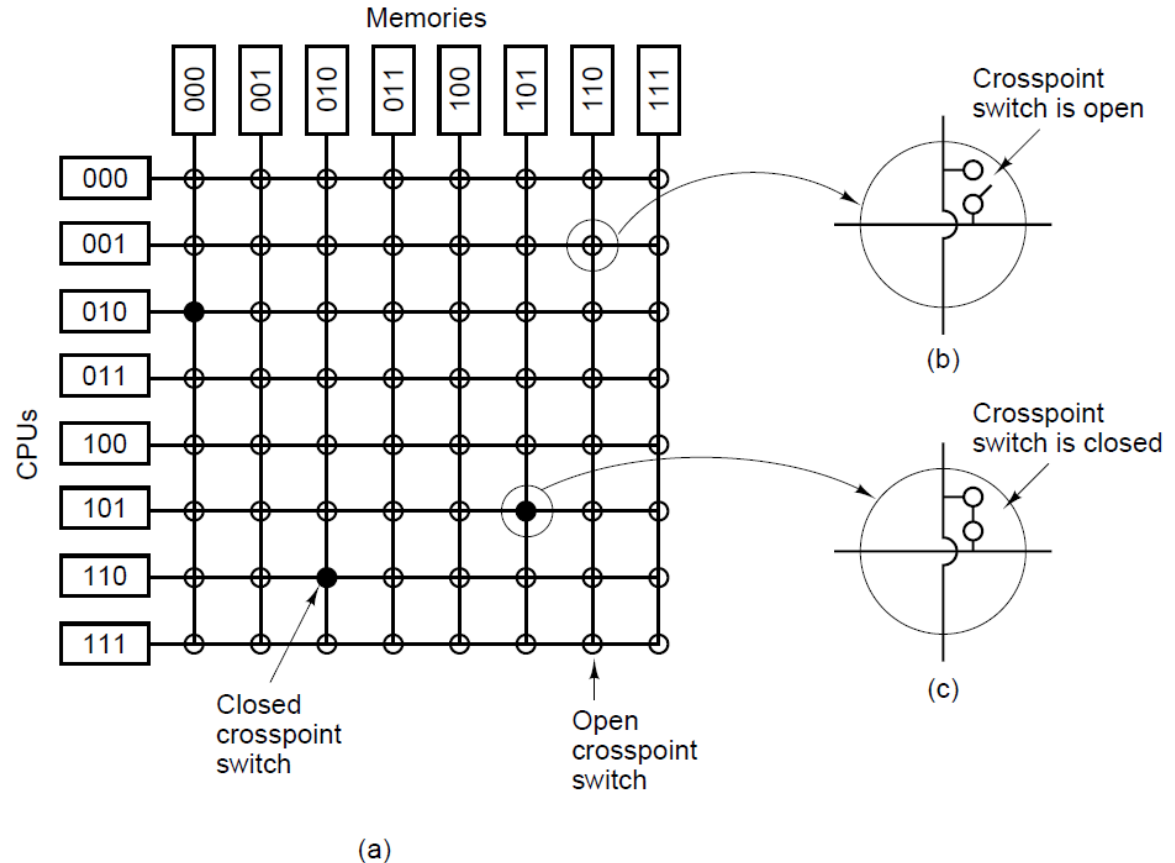
(a) Without caching. (b) With caching. (c) With caching and private memories.

UMA Multiprocessor (tt)



UMA multiprocessors using Crossbar Switches

- ☐ Tăng số lượng processor trong hệ thống
- ☐ Hệ thống kết nối trong viễn thông
- ☐ **Nonblocking network**
- ☐ Không cần dự trữ kết nối trước
- ☐ crossbar switch tăng khi số lượng processor tăng



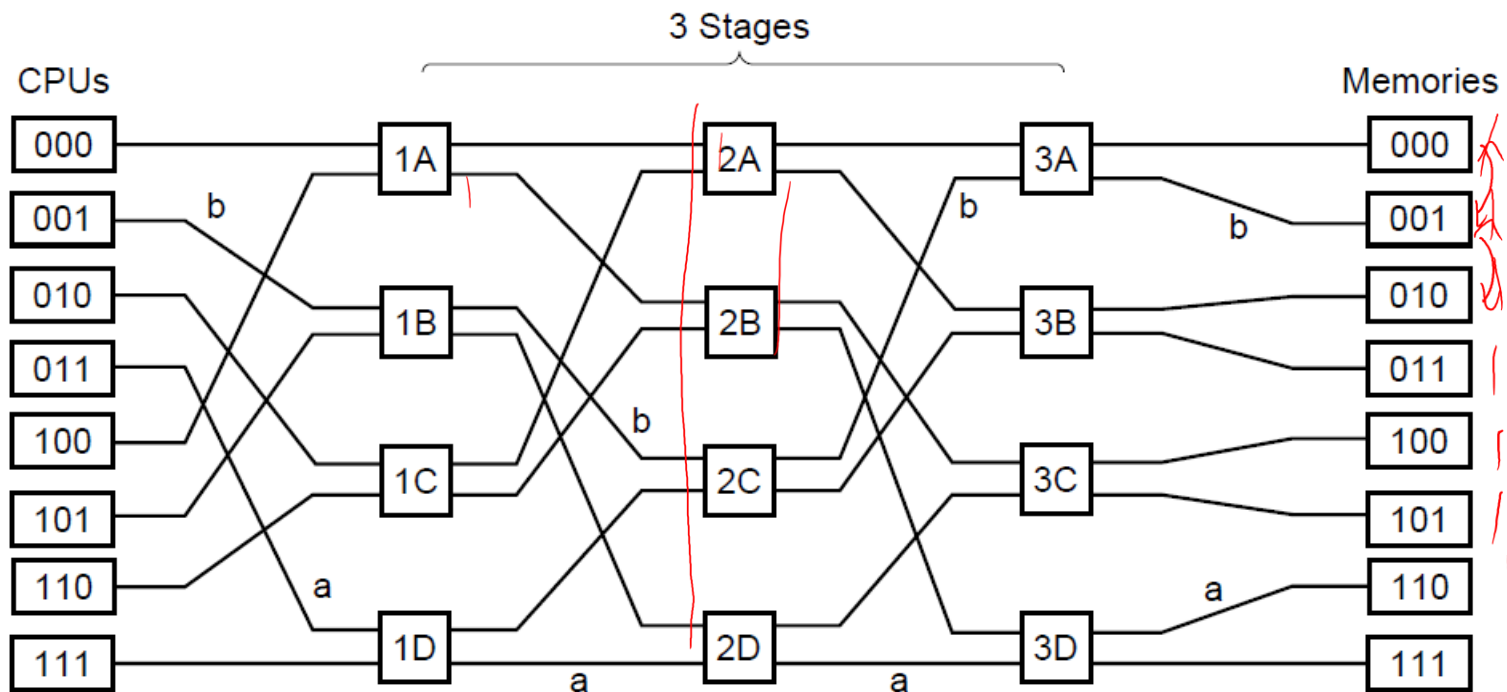
- (a) An 8×8 crossbar switch. (b) An open crosspoint.
(c) A closed crosspoint.

UMA Multiprocessor (tt)



UMA multiprocessors using Multistage Switching Networks

- ☐ Blocking network
- ☐ Interleaved memory
- ☐ Fully connected



An omega switching network

NUMA Multiprocessor



❑ Hạn chế của kiến trúc UMA:

- Bus-based: số lượng processor thấp
- Crossbar và multistage: tăng chi phí phần cứng

➔ **NUMA multiprocessors**

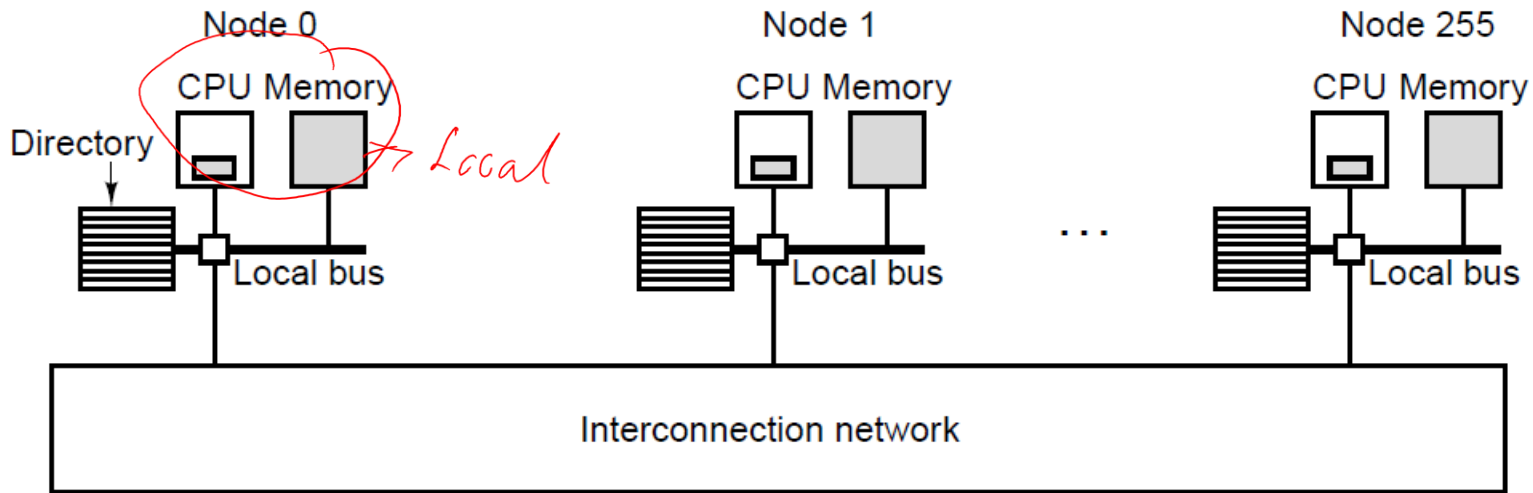
❑ Đặc điểm của NUMA:

- Có một không gian địa chỉ đơn cho tất cả các CPU
- Truy xuất đến bộ nhớ xa thông qua lệnh LOAD và STORE
- Truy xuất bộ nhớ xa thì chậm hơn là bộ nhớ cục bộ

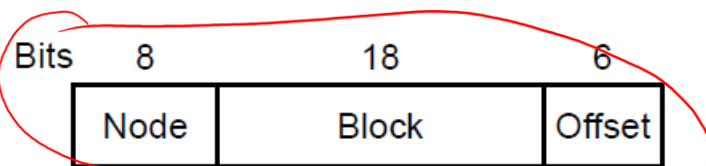
CC-NUMA



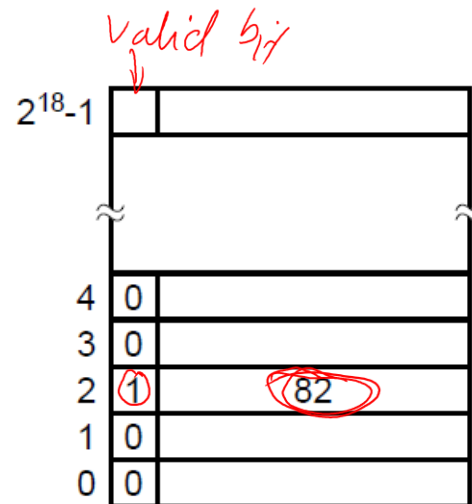
Cache Coherence – Non-uniform memory access



(a)



(b)



(c)

- (a) A 256-node directory-based multiprocessor.
 (b) Division of a 32-bit memory address into fields.
 (c) The directory at node 36.



3. Một số vấn đề với Multiprocessor

- ☐ Hệ điều hành
- ☐ Đồng bộ hóa
- ☐ Định thời CPU

Các dạng tổ chức hệ điều hành

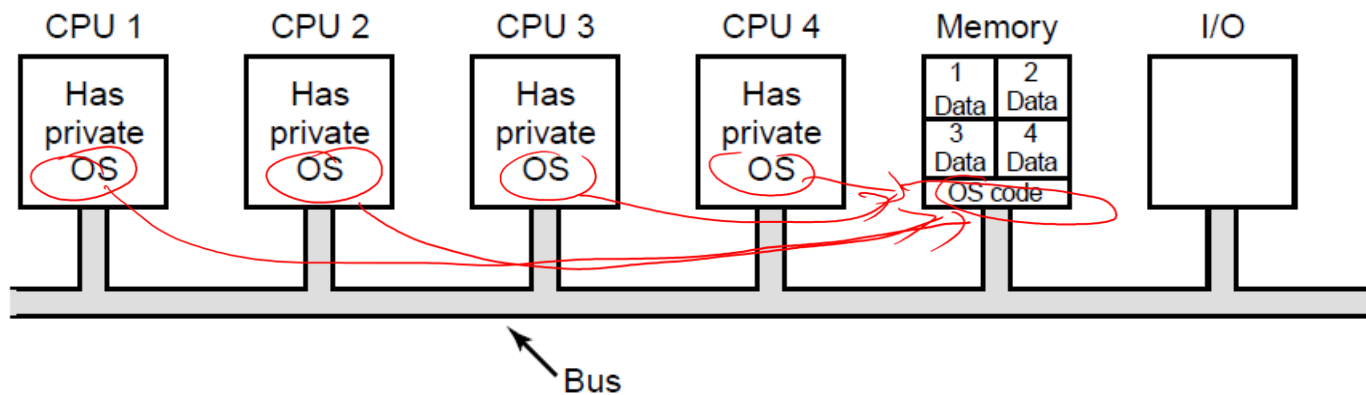


- ☐ Mỗi CPU có hệ điều hành riêng
- ☐ Dạng Master-Slave
- ☐ Dạng đối xứng (Symmetric Multiprocessor)

Mỗi CPU có hệ điều hành riêng



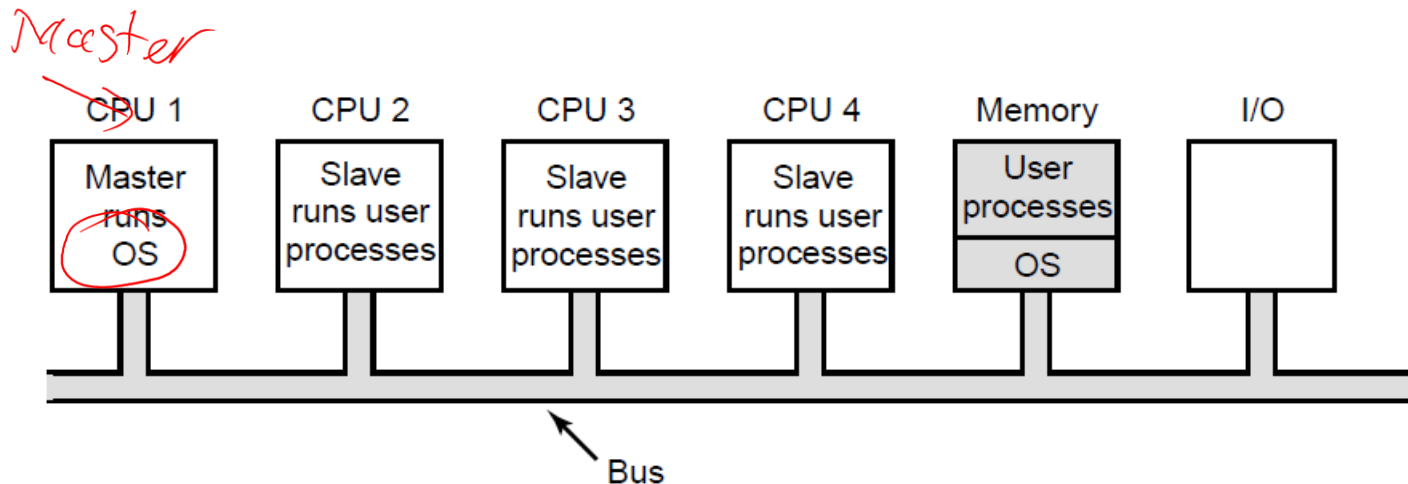
- ❑ Mỗi CPU có một bản sao của HĐH
- ❑ Chia sẻ bus, bộ nhớ và IO
- ❑ Mỗi CPU được cấp phát một vùng nhớ riêng trên memory



Master-Slave Multiprocessor



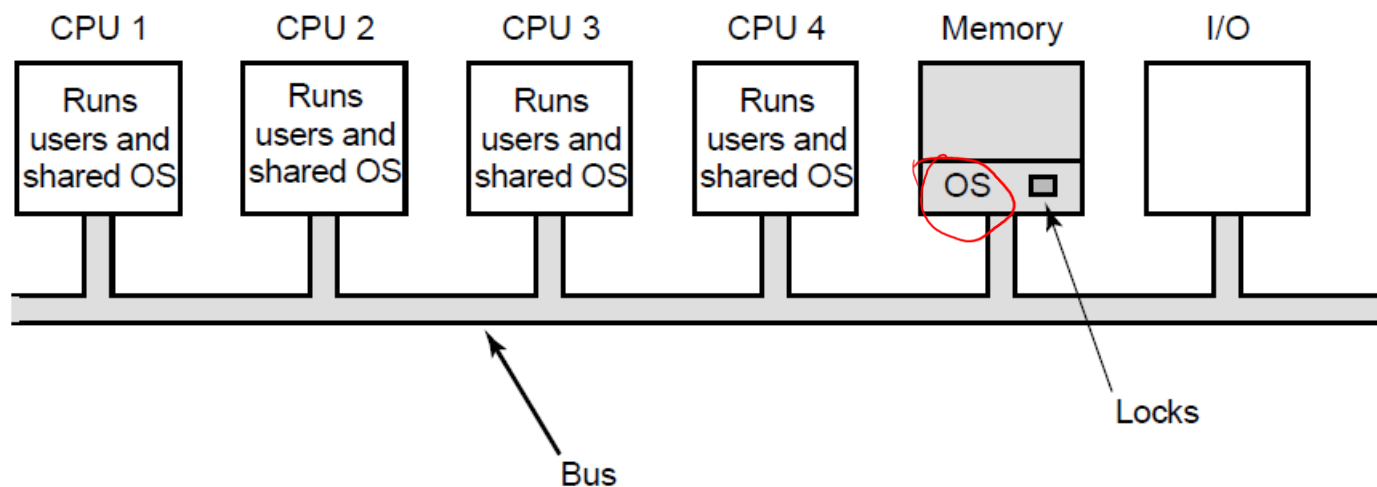
- ☐ Một CPU là Master, các CPU còn lại là Slave
- ☐ Master CPU thực thi hệ điều hành, phân chia tiến trình, cấp phát tài nguyên...
- ☐ Slave CPU thực thi các tiến trình người dùng
- ☐ Chia sẻ bus, bộ nhớ và IO



Symmetric Multiprocessor



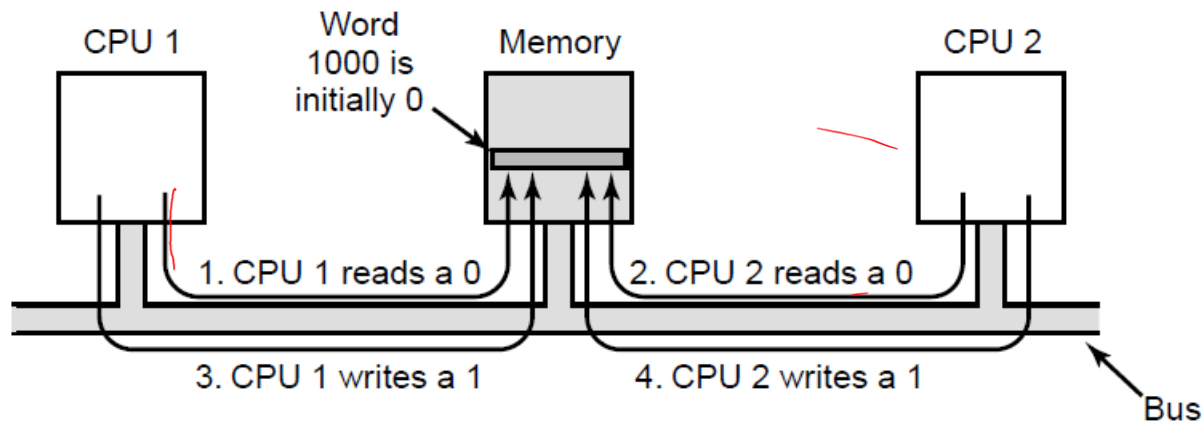
- ❑ Một bản sao của HĐH được lưu trên bộ nhớ
- ❑ Bất cứ CPU nào cũng có thể thực thi HĐH
- ❑ Khi một CPU thực hiện lời gọi hệ thống, CPU cần thực hiện bẫy (khóa) kernel
- ❑ Yêu cầu cơ chế đồng bộ khi sử dụng lời gọi hệ thống



Đồng bộ hóa trên Multiprocessor



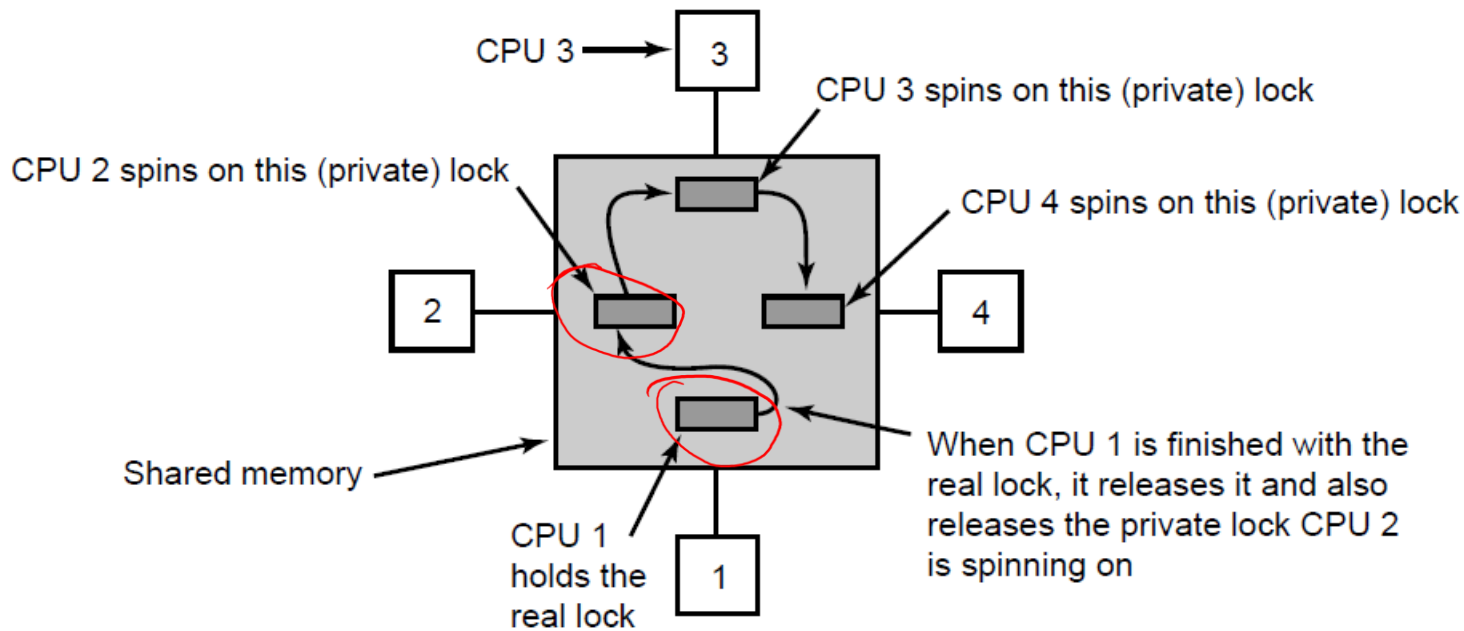
- ☐ Các bảng nhớ và vùng tranh chấp cần được bảo vệ
- ☐ Cơ chế Mutex
- ☐ Việc đồng bộ được đảm bảo cho tất cả các CPU
- ☐ TSL (Test and Set Lock) instruction?



Đồng bộ với nhiều khóa



Sử dụng nhiều khóa để tránh việc tranh chấp cache



Định thời đa vi xử lý



- ❑ Câu hỏi định thời:

1. Tiến trình nào cần được thực thi tiếp theo?
2. Tiến trình cần được thực thi trên CPU nào?

- ❑ Sự phụ thuộc của các tiến trình (ứng dụng)

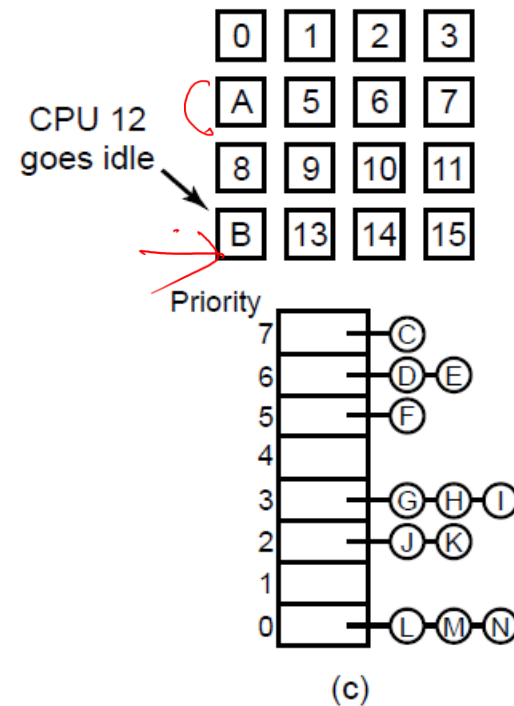
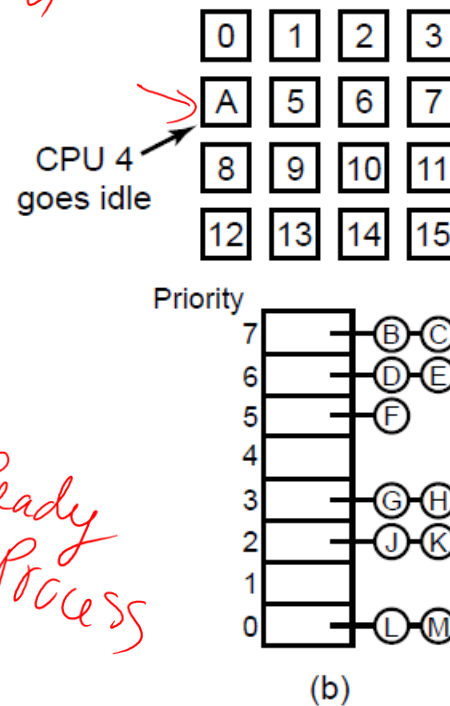
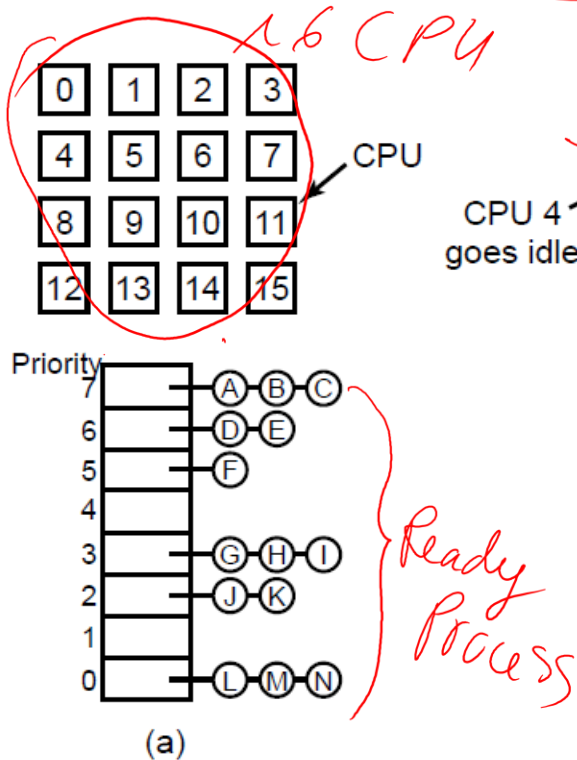
- ❑ Chiến lược định thời:

- ❑ Chia sẻ thời gian (time sharing)
- ❑ Chia sẻ không gian (space sharing)

Time sharing



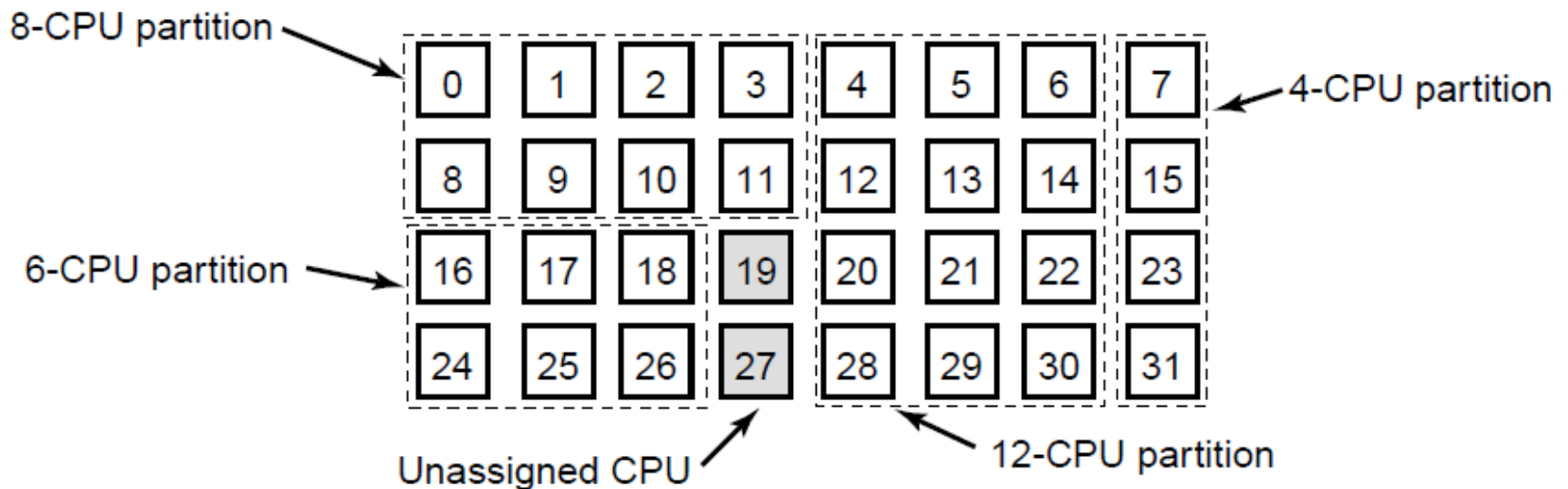
- ❑ Các tiến trình độc lập nhau
 - ❑ Điểm mạnh: tự cân bằng tải
 - ❑ Cấp phát “thông minh” cho lần thực thi tiếp theo?
- **affinity scheduling**



Space sharing



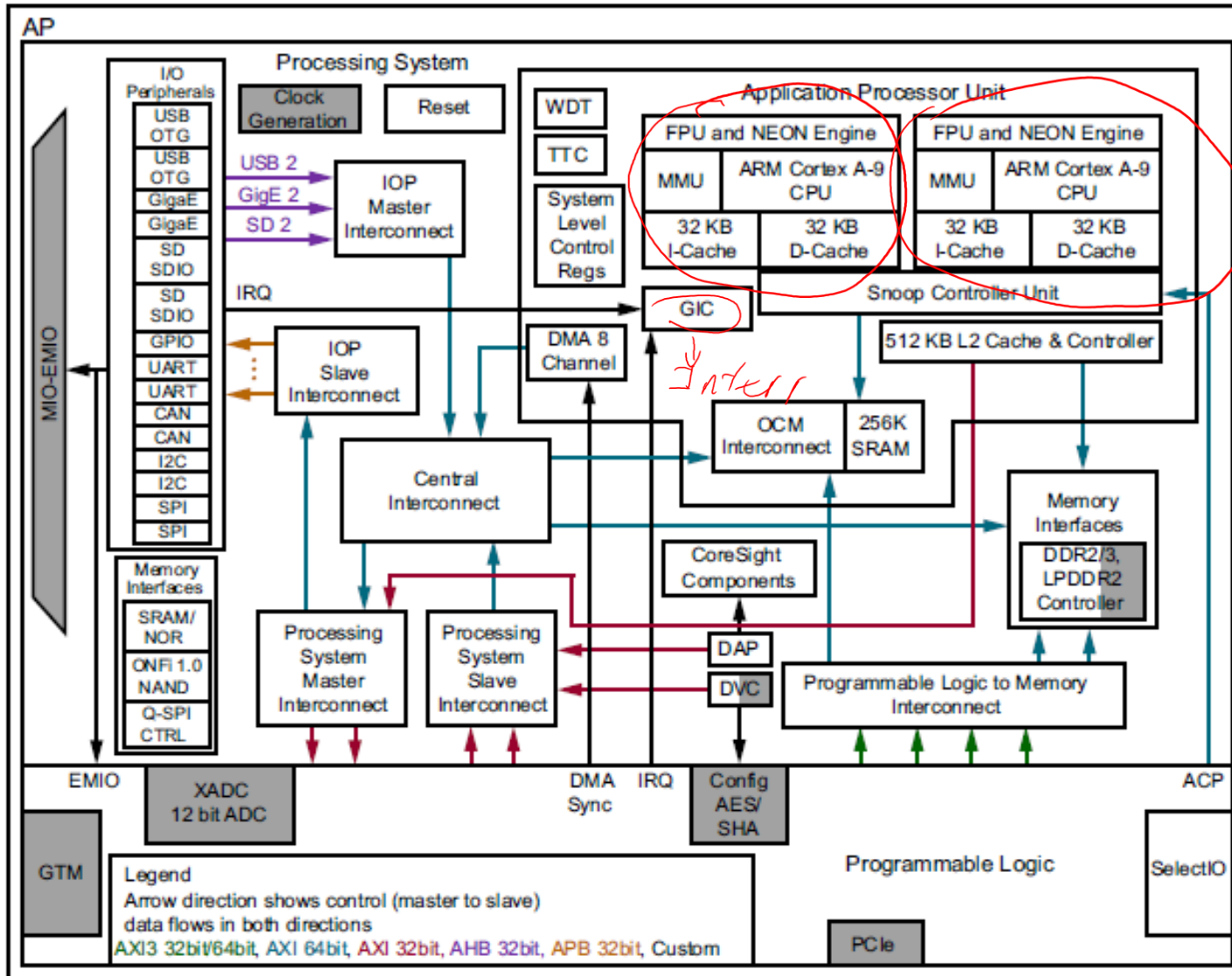
- ☐ Tiến trình có liên quan đến nhau
- ☐ Xử lý đa luồng
- ☐ Cấp phát CPU theo nhóm
- **Processor Partition**





3. Ví dụ Multiprocessor SoC

Zynq-7000 All Programmable SoC



Tính năng của Zynq-7000 SoC



□ Tham khảo tài liệu:

- ***Zynq-7000 SoC Technical Reference Manual***
UG585 Version 1.12.2 July 1, 2018
- ***Zynq-7000 All Programmable SoC Software***
Developers Guide, UG821 (v12.0) September
30, 2015



UIT-HCM



Thank You !